



전자전기컴퓨터설계실험II

실험 전 보고서

순차논리회로 실습

실험 11-18

학과	전자전기컴퓨터공학부
이름 / 학번	조성현 / 2023440129
작성일	2026년 09월 21일
대상	업 / 다운 카운터 · 클록 분주 · 레지스터 · 쉬프트 레지스터 · PISO · Moore 상태머신 · Mealy 상태 머신 · 7세그먼트 자리 스캔
검증환경	VS Code 1.137.0 / Icarus Verilog 13.0 / VaporView / Vivado 2026.1

소스 식별

GitHub 계정: a40093632-ai

GitHub 저장소: <https://github.com/a40093632-ai/ece2-lab2>

제출 태그: [lab2-pre-v1](#)

커밋 해시: [72f002c5c7dfcb225a4ff0d72dda3f3cb943d592](#)

```

tag lab2-pre-v1
tagger: a40893632-a1 <a40893632@gmail.com>
taggerDate: Mon Sep 21 13:09:22 2026 +0900
LAB2 pre-lab source and evidence baseline
commit 72f002c5c7d4c225a4ff6d73d4c3f3cb9d3d592 (HEAD -> main, tag: lab2-pre-v1, origin/main)
Author: a40893632-a1 <a40893632@gmail.com>
AuthorDate: Mon Sep 21 13:08:21 2026 +0900
Commit: a40893632-a1 <a40893632@gmail.com>
CommitDate: Mon Sep 21 13:08:21 2026 +0900

Add LAB2 integrated simulation evidence
PS D:\Engineering\Verilog_Lab\ece2-lab2>

```

그림 1: GitHub 제출 태그 및 기준 커밋 확인

1. 실험 목적 및 범위

실험 목적

Lab2의 목적은 Verilog HDL을 이용하여 클록과 내부 상태를 가지는 순차논리회로를 설계하고, 상승 엣지에서 입력과 이전 상태에 따라 다음 상태가 결정되는 과정을 이해하는 것이다. 실험에서 주어진 8가지 회로를 각각 구현해 보고, Verilog의 PASS 로그와 VaporView 파형으로 예상 결과와 실제 결과를 비교하고자 한다.

실험 범위

	회로	목적
11	4비트 업 / 다운 카운터	Enable과 방향 입력에 따른 증감 및 순환 확인
12	클록 분주	분주 출력과 1클록 생성 확인
13	레지스터	레지스터의 특성 확인
14	시프트 레지스터	직렬 입력과 4비트 상태 이동 확인
15	PISO	병렬 데이터의 직렬 변환 확인
16	무어 머신	현재 상태만으로 결정되는 출력 확인
17	밀리 머신	현재 상태와 입력에 의해 결정되는 출력 확인
18	7세그먼트 자리 스캔	자리 선택, Blank 구간 및 세그먼트 패턴 확인
18A	8모드 통합회로	모드 선택 및 전체 회로 파악

2. 관련 이론

순차논리회로와 상승 엣지

순차논리회로의 출력은 현재 입력뿐 아니라 플립플롭에 저장된 이전 상태에 의해 결정된다. 이번 실험의 상태 레지스터는 `always @(posedge clk)`에서 갱신되므로 입력은 상승 엣지 구간에서 반영된다.

Nonblocking assignment

순차회로에서는 `<=`를 사용하므로 동일한 상승 엣지에서

오른쪽 신은 갱신 전 상태를 참조하는 동작을 한다. 따라서 register pair에서 load와 transfer가 동시에 1이면 stored에는 새 데이터가 저장되지만 value에는 갱신 전 값이 전달된다. 이와 반대의 동작을 하는 게 blocking이다

Moore와 Mealy 상태기계

유한상태기계(FSM)는 현재 상태와 입력에 따라 다음 상태와 출력을 결정하는 순차논리회로이다. Moore 상태기계는 출력이 현재 상태에 의해서만 결정된다. 따라서 입력이 바뀌어도 상태가 클록 에지에서 변경되기 전까지 출력이 유지되어 비교적 안정적이다[1]. 반면 Mealy 상태기계는 출력이 현재 상태와 입력에 의해 결정된다. 상태는 클록 에지에서 변경되지만 출력은 입력 변화에 즉시 반응할 수 있어 Moore 방식보다 빠르게 동작할 수 있다. 다만 입력의 순간적인 변화도 출력에 반영될 수 있으므로 입력 신호의 안정성을 고려해야 한다[2].

3. 개발 환경

개발환경

항목	사용 환경
운영체제	Windows 11 64-bit
코드 작성	VS Code 1.137.0
HDL Simulator	Icarus Verilog 13.0
파형 확인	VaporView
FPGA Part	Xc7s75fgga484-1
실행 Script	Python 3.14.7
FPGA 도구	AMD Vivado 2026.1

파일의 역할

Src 폴더에는 각 순차논리회로의 기능을 구현한 RTL 파일과 FPGA 보드 입출력을 연결하는 top 모듈이 있다. 예시로 `input_frontend.v`는 버튼과 스위치 입력을 동기화하고 버튼 입력을 한 클록 폭의 신호로 변환한다.

sim 폴더의 테스트벤치는 회로에 입력을 인가하고 예상값과 실제 출력을 비교하여 PASS 또는 FAIL을 판단하며, 파형 확인을 위한 `wave.vcd`를 생성한다. `simulation.json`은 시뮬레이션에 사용할 RTL-테스트벤치 파일과 `simulation top`을 지정한다.

Vivado 등록 항목

실 험	Design Top	Simul Top	XDC
11	lab2_counter	tb_counter4	lab2_counter.xdc
12	lab2_clock_divider	tb_clock_divider	lab2_clock_divider.xdc
13	lab2_register	tb_register_pair	lab2_register.xdc
14	lab2_shift_regist	tb_shift_registe	lab2_shift_register.xdc

	er	r4	xdc
15	lab2_piso	tb_piso4	lab2_piso.xdc
16	lab2_moore	tb_moore_cycle	lab2_moore.xdc
17	lab2_mealy	tb_mealy_toggle	lab2_mealy.xdc
18	lab2_segment_scan	tb_segment_scan8	lab2_segment_scan.xdc
18 A	lab2_integrated	tb_lab2_integrated	lab2_integrated.xdc

4. 실험 11 · 업 / 다운 카운터

핵심 내용

·코어: counter4.v

·입력: clk, rst, enable, down

·출력: value[3:0]

·정상 로그: LAB2_PASS counter4 checks=36

핵심 코드

```
always @(posedge clk) begin
    if (rst)
        value <= 4'b0000;
    else if (enable) begin
        if (down)
            value <= value - 4'b0001;
        else
            value <= value + 4'b0001;
    end
end
```

그림 2 : 실험 11 Verilog 핵심 코드

클록의 상승 에지마다 4비트로 표현된 1을 더하거나 빼주는 코드.

예상 결과

Rst	Enable	Down	다음 value
1	X	X	0
0	0	X	전 값 유지
0	1	0	Value+1, F 다음은 0
0	1	1	Value-1, 0 다음은 F

Rst가 1이면 enable과 down의 값에 관계없이 상승 에지에서 value가 0으로 초기화된다. Rst=0이고 enable=0이면 대입문이 실행되지 않으므로 기존 value를 유지한다. 그 외에도 동일하게 표와 같이 예상할 수 있다.

결과 설명

```
Run C:\msys64\mingw64\bin\vsrv.exe -N D:\Engineering\Verilog_Lab\lab2\lab2_01_counter\build\sim\run-84ff0f75564546d8f59953f308a08a7\sim.vvp
INFO: dumpfile wave.vcd opened for output.
LAB2_PASS counter4 checks=36
D:\Engineering\Verilog_Lab\lab2\lab2_01_counter\sim\tb_counter4.sv:181: %finish called at 355000 (ps)
```

그림 3 : 실험 11 시뮬레이션 실행 결과(PASS)

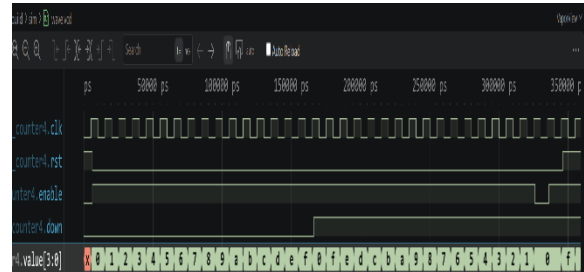


그림 4 : 실험 11 입출력 파형

초기 value의 X는 첫 reset 상승 에지 이전의 미정 상태이며, rst가 인가된 첫 상승 에지에서 0으로 초기화되었다. 이후 enable=1, down=0인 구간에서 value가 순서대로 증가하였고, 나머지 결과도 예상 결과와 일치한다.

5. 실험 12 · 클록 분주

핵심 내용

·코어: clock_divider.v

·출력: divided, tick

·정상 로그: LAB2_PASS clock_divider checks=129

핵심 코드

```
if (count == DIVISOR - 1)
    count <= 0;
else
    count <= count + 1'b1;

if (count == DIVISOR/2-1)
    divided <= 1;
else if (count == DIVISOR - 1)
    divided <= 0;
```

그림 5 : 실험 12 Verilog 핵심 코드

카운터를 이용해 입력 클록을 분주하고, 한 주기가 끝날 때 tick을 한 클록 동안 출력하는 코드.

예상 결과

조건	Divided 예상값	Tick 예상값
Rst=1	0	0
DIVISOR=10, cycle%10=0~4	0	0

```
DIVISOR=10, cycle%10=5~8      1      0
DIVISOR=10, cycle%10=9      1      1
```

Divisor=10인 경우 divided는 입력 클럭 다섯 주기 동안 0, 다음 다섯 주기 동안 1이 되므로 입력 클럭 주기의 10배인 분주 신호가 예상된다.

결과 설명

```
Run C:\msys64\mingw64\bin\vvp.EXE -I D:\Engineering\Verilog_Lab\lab2\lab2_02_clock_divider\build\sim-run-808f15758f94f459430fc465693584\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS clock_divider checks=129
D:\Engineering\Verilog_Lab\lab2\lab2_02_clock_divider\sim\th_clock_divider.sv:86: $finish called at 436888 (ps)
```

그림 6: 실험 12 시뮬레이션 실행 결과(PASS)

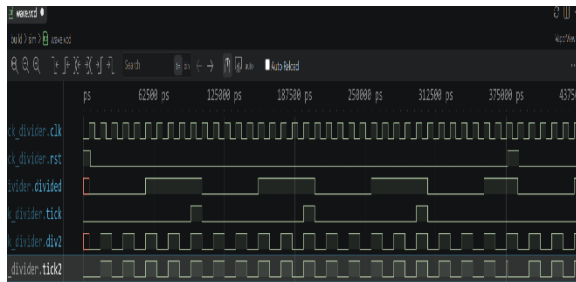


그림 7: 실험 12 입출력 파형

파형에서 divisor=10인 divided는 약 50ns 동안 0, 다음 50ns 동안 1을 반복하였다. Tick은 각 10클럭 구간의 끝에서 한 클럭 동안만 1이 되었다. 또한 약 375ns 부근에서 rst가 인가되자 모든 출력이 0으로 초기화되었으며, 이는 예상 결과와 일치한다.

6. 실험 13 · 레지스터: 데이터 저장과 이동

핵심 내용

·코어: register_pair.v

·입력: load, transfer, data_in[3:0]

·출력: stored[3:0], value[3:0]

·정상로그: LAB2_PASS register_pair checks=7

핵심 코드

```
if (load)
    stored <= data_in;

if (transfer)
    value <= stored;
```

그림 8: 실험 13 Verilog 핵심 코드

load에서 입력을 저장하고, transfer에서 저장값을 출력 레지스터로 전달하는 코드

예상 결과

Load	Transfer	Stored 다음 값	Value 다음 값
0	0	유지	유지
1	0	Data_in	유지
0	1	유지	이전 값
1	1	Data_in	이전 값

load=1이면 data_in이 stored에 저장되고, transfer=1이면 저장되어 있던 stored가 value로 전달될 것으로 예상된다. load와 transfer가 동시에 1인 경우에는 nonblocking assignment 때문에 stored에는 새로운 data_in이 저장되지만 value에는 상승 에지 이전의 stored가 전달된다.

결과 설명

```
Run C:\msys64\mingw64\bin\vvp.EXE -I D:\Engineering\Verilog_Lab\lab2\lab2_03_register\build\sim-run-808f15758f94f459430fc465693584\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS register_pair checks=7
D:\Engineering\Verilog_Lab\lab2\lab2_03_register\sim\th_register.sv:124: $finish called at 66888 (ps)
```

그림 9: 실험 13 시뮬레이션 실행 결과(PASS)

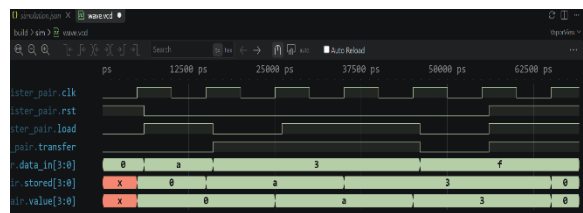


그림 10: 실험 13 입출력 파형

리셋 후 stored와 value는 모두 0이 되었다. data_in=A, load=1인 상승 에지에서는 stored만 A로 변경되고 value는 0을 유지하였다. 이후 data_in=3, transfer=1인 에지에서 value는 현재 data_in인 3이 아니라 저장되어 있던 A가 되었다. 이는 예상한 결과와 동일하고, load와 transfer 모두 1인 경우도 일치하였다.

7. 실험 14 · 시프트 레지스터

핵심 내용

·코어: shift_register4.v

·다음 상태: {serial_in, value[3:1]}

·정상 로그: LAB2_PASS shift_register4 checks=8

핵심 코드

```
if (rst)
    value <= 4'd0;
else if (enable)
    value <= {serial_in, value[3:1]};
```

그림 11: 실험 14 Verilog 핵심 코드

매 상승 에지마다 기존 값을 오른쪽으로 이동시키고 serial_in을 최상위 비트에 삽입하는 코드.

예상 결과

Enable	Serial_in	이전 값	예상 값
1	1	0000	1000(8)
1	0	1000	0100(4)
0	1	0100	0100(4)

enable=1인 상승 에지마다 기존 value[3:1]이 한 비트씩 LSB 방향으로 이동하고 serial_in이 새 MSB로 들어올 것으로 예상된다. enable=0이면 serial_in이 변하더라도 value는 유지된다.

결과 설명

```

RM C:\msys64\mingw64\bin\vvp.exe -I D:\Engineering\Verilog_Lab\lab2\lab2_04_shift_register\build\sim\run-c8e40ed534c66b70395f99723ad\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS shift_register4 checks=4
D:\Engineering\Verilog_Lab\lab2\lab2_04_shift_register\sim\tb_shift_register4.sv:101: $finish called at 10000 (ps)

```

그림 12: 실험 14 시뮬레이션 실행 결과(PASS)



그림 13: 실험 14 입 출력 파형

구간을 살펴보면 enable=0일 때 serial_in=1이어도 value=4가 유지되었다. enable이 다시 1이 된 후에는 value가 A, 5, 2, 1, 0 순으로 변화였다. 이는 이전 값(value)이 0100일 때 예상결과의 동작과 동일하게 움직였다.

8. 실험 15 · PISO

핵심 내용

- 코어: piso4.v
- load 우선순위: rst > load > enable > hold
- 직렬 출력: serial_out=value[3]
- 정상 로그: LAB2_PASS piso4 checks=114

핵심 코드

```

assign serial_out = value[3];

always @(posedge clk) begin
    if (rst)
        value <= 0;
    else if (load)
        value <= data_in;
    else if (enable)
        value <= {value[2:0], 1'b0};
end

```

그림 14: 실험 15 Verilog 핵심 코드

병렬 데이터를 저장한 뒤 MSB부터 직렬로 출력하는 코드.

예상 결과

단계	Load	Enable	예상 value	예상 Serial_out
병렬저장	1	1	1010(A)	1
Hold	0	0	1010(A)	1
1회 shift	0	1	0100(4)	0

위의 예상 결과표는 data_in=A(1010)인 경우를 예시로 든 것이다. 우선 순위가 load가 enable 보다 높으므로 데이터는 value에 병렬 저장될 것이고 serial_out은 value의 MSB에 연결되므로 다음과 같은 출력을 가질 것이다.

결과 설명

```

RM C:\msys64\mingw64\bin\vvp.exe -I D:\Engineering\Verilog_Lab\lab2\lab2_05_piso\build\sim\run-5348ad0ba37442c94c93c6f0767f716\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS piso4 checks=114
D:\Engineering\Verilog_Lab\lab2\lab2_05_piso\sim\tb_piso4.sv:75: $finish called at 97000 (ps)

```

그림 15: 실험 15 시뮬레이션 실행 결과(PASS)

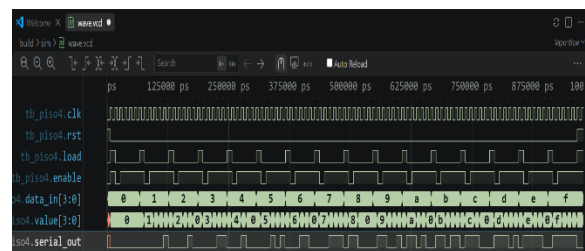


그림 16: 실험 15 입 출력 파형

파형 사진을 보면 데이터는 0부터 F까지 인가되었다. 각 데이터 구간의 load 펄스에서 value가 병렬 입력 값으로 변경되었고, enable=0인 구간에서는 값이 유지되었다. 이후 enable=1인 상승 에지마다 value가 왼쪽으로 이동한 것을 보면 예상한 병렬 입력 및 MSB 우선 직렬 출력 동작과 일치한다.

9. 실험 16 · Moore 상태 머신

핵심 내용

·상태: S0=00, S1=01, S2=10

·전이: S0→S1→S2→S0

·전이 조건: enable && advance

·정상 로그: LAB2_PASS moore_cycle checks=23

핵심 코드

```
case (value)
    S0: next_state = S1;
    S1: next_state = S2;
    S2: next_state = S0;
    default: next_state = S0;
endcase

always @(posedge clk) begin
    if (rst)
        value <= S0;
    else if (enable && advance)
        value <= next_state;
end
```

그림 17: 실험 16 Verilog 핵심 코드

출력은 현재 상태에 의해서만 결정되며 S0→S1→S2→S0으로 순환하는 코드.

예상 결과

현재상태	Enable	Advance	예상 다음 상태
S0(0)	1	1	S1(1)
S1(1)	1	1	S2(2)
S2(2)	1	1	S0(0)

무어는 오로지 현재 상태에 의해 출력이 결정된다. Enable과 advance가 1일 때 전이가 발생하므로 위와 같은 결과가 나온다.

결과 설명

```
Run C:\msys64\mingw64\bin\vp.exe -N D:\Engineering\Verilog_Lab\lab2\lab2_06_moore\build\sim-run-3780c9a277094e4c0765fe61f902c740\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS moore_cycle checks=23
D:\Engineering\Verilog_Lab\lab2\lab2_06_moore\sim\to_moore_cycle.sv:182: $finish called at 228000 (lps)
```

그림 18: 실험 16 시뮬레이션 실행 결과(PASS)

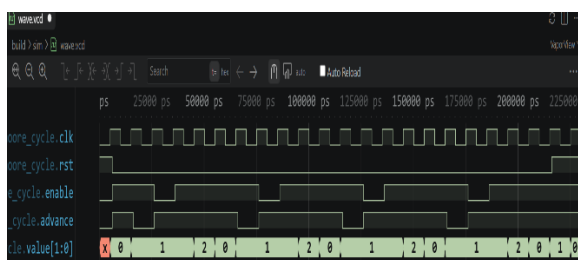


그림 19: 실험 16 입출력 파형

파형에서 value는 0, 1, 2, 0 순서로 반복되었다. advance만

변경된 구간에서는 상태가 즉시 바뀌지 않았고, enable과 advance가 모두 1인 상승 에지에서만 다음 상태로 전이하였다. advance=0 또는 enable=0인 구간에서는 현재 상태를 유지하였다. 이는 전형적인 무어 머신의 동작이다.

10. 실험 17 · Mealy 상태 머신

핵심 내용

·상태: state

·입력: bit_in / ·출력: value[1:0]

·정상 로그: LAB2_PASS mealy_toggle checks=11

핵심 코드

```
else if (enable && bit_in)
    state <= ~state;
end

always @(*) begin
    if (!bit_in)
        value = 2'b00;
    else if (state)
        value = 2'b01;
    else
        value = 2'b10;
end
```

그림 20: 실험 17 Verilog 핵심 코드

현재 상태와 입력 bit_in을 함께 사용해 출력을 즉시 결정하는 코드.

예상 결과

현재 state	Enable	Bit_in	예상 value	상승 후 state
0	0	0	00(0)	0
0	0	1	10(2)	0
0	1	1	10→01	1
1	1	0	00(0)	1

bit_in=0이면 state에 관계없이 value=0이 될 것으로 예상된다. bit_in=1이면 state=0에서 value=2, state=1에서 value=1이 된다. Value는 에지 구간과 상관없이 bit_in의 변화가 우선적으로 영향을 끼치므로 전형적인 밀리 머신 동작으로 예상할 수 있다.

결과 설명

```
Run C:\msys64\mingw64\bin\vp.exe -N D:\Engineering\Verilog_Lab\lab2\lab2_07_mealy\build\sim-run-50bc913b91f44990a211c9371d6615\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB2_PASS mealy_toggle checks=11
D:\Engineering\Verilog_Lab\lab2\lab2_07_mealy\sim\to_mealy_toggle.sv:128: $finish called at 67000 (lps)
```

그림 21: 실험 17 시뮬레이션 실행 결과(PASS)



그 림 22 : 실험 17 입출력 파형

파형을 살펴보면 state=0인 구간에서 bit_in이 1로 변경되자 상승 에지가 아니어도 value가 2로 바뀌었다. 이외에도 enable=1 일 때 state 값도 반전되면서 value가 1이 되었다. 이처럼 에지 관계없이 bit_in의 값에 출력이 바뀌는 것은 전형적인 밀리 동작과 일치한다.

11. 실험 18 · 7세그먼트 자리 스캔

핵심 내용

·코어: segment_scan8.v

·입력 데이터: digits[31:0]

출력: select[7:0], segments[7:0], index[2:0]

정상 로그: LAB2_PASS segment_scan8 checks=194

핵심 코드

```
always @(posedge clk) begin
    if (rst) begin
        index <= 0;
        blank <= 1;
    end else if (enable) begin
        blank <= ~blank;

        if (!blank)
            index <= index + 1'b1;
    end
end
```

그림 23: 실험 18 Verilog 핵심 코드

Enable이 활성화되면 blank를 반전시키고 표시가 끝난 시점에 index를 증가시키는 코드 부분.

예상 결과

Index	선택 숫자	예상 select	예상 segment
0	0	01	FC
1	1	02	60
2	2	04	DA
3	3	08	F2

Digits=7654_3210이라 가정하면 index는 데이터의 비트 위치를 가리키고, 그 비트 값에 맞는 세그먼트 값을 다음과 같이 예상할 수 있다. 각 자리의 active 구간 사이에는

select=00인 blank 구간이 삽입된다. enable=0이면 blank
와 index가 변경되지 않아 현재 출력이 유지된다.

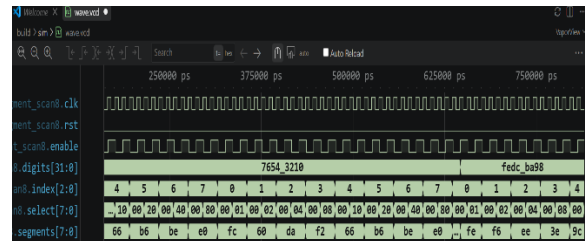
결과 설명

```

C:\msys4\mingw64\bin>wc E:\1-D:\Engineering\Verilog_Lab\ex1-lab1\lab1_00_segment_scan\build\sim-run-320464a40a46f6a7f502b7e9d663\sim.vvp
VCD info: dumpfile wave.vcd opened for output.
LAB1_PASS segment_scan checks=194
D:\Engineering\Verilog_Lab\ex1-lab1\lab1_00_segment_scan\sim\test\segment_scan.vv35: ffinish called at 1366000 (ps)

```

그림 24: 실험 18 시뮬레이션 실행 결과(PASS)



그 림 25 : 실험 18 입출력 파형

파형을 분석해보아도 처음 입력인 7654_3210에서 예상과 같은 세그먼트 값이 출력되었다. 각 유효 select 사이에는 select=00인 blank 구간도 나타났고, 출력이 유지되는 것을 확인했다.

12. 실험 18A · 8모드 통합 회로

핵심 내용

·설계 파일: lab2_integrated.v

.테스트 벤치: tb_lab2_integrated.sv

·입력: clk, rst, mode_button, step_button, sw[7:0]

·출력: led[7:0], LCD 및 7세그먼트 제어 신호

정상로그: LAB2 INTEGRATED PASS modes=8 checks=2848

핵심 코드

```
always @(posedge clk) begin
    if (reset) mode <= 0;
    else if (mode_press) mode <= mode + 1'b1;
end
// Each new mode starts from its initial state
wire circuit_reset = reset || mode_press;
```

```
always @* begin
    led = 0;
    case (mode)
        0: led = {4'b0, count};
        1: led = {3'b0, tick, divided, div50, div10, div2};
        2: led = {stored, registered};
        3: led = {4'b0, shifted};
        4: led = {parallel, 3'b0, serial_out};
        5: led = {6'b0, moore_value};
        6: led = {5'b0, mealy_state, mealy_value};
        7: led = {5'b0, scan_index};
    endcase
end
```

Mode_press가 발생할 때마다 3비트 mode가 증가하여 8개 모드가 순환하는 코드. 또한 case문에서 현재 mode에 해당하는 회로의 결과만 led로 출력하는 코드.

그림 26: 실험 18A Verilog 핵심 코드

예상 결과

위의 핵심 코드에 적힌 case 구문에서 볼 수 있듯이 mode 별로 순서대로 앞선 8개의 순차논리회로 동작이 일어날 것이다.

결과 설명

```

C:\msys64\mingw64\bin\gcc.exe -D:\Engineering\Verilog_Lab\ece2-lab2\lab2_08a_integrated\build\sin-run-2b555d6ba7449480238e-f06a59807\sin.vvp
VC info: dumpfile wave.vcd opened for output.
LAB2_INTEGRATED_PASS mode=0 checks=0x0
D:\Engineering\Verilog_Lab\ece2-lab2\lab2_08a_integrated\sin\lab2_integrated.sv:129: $finish called at 3435000 (ps)
  
```

그림 27: 실험 18A 시뮬레이션 실행 결과(PASS)

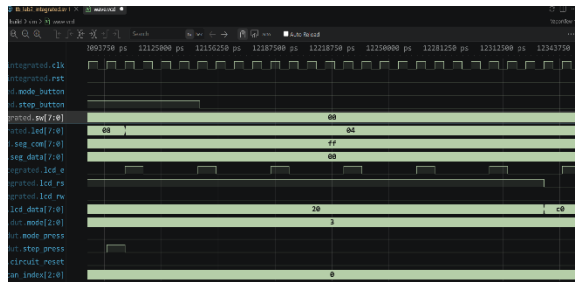


그림 28: 실험 18A 입 출력 파형

파형은 mode=3인 Shift Register 구간으로, step_press가 한 클럭 동안 발생한 뒤 LED 값이 08에서 04로 변경되었다. mode 7이 아니므로 seg_com=FF, seg_data=00으로 7 세그먼트가 비활성화된 것도 확인하였다.

13. 정상 / 변경 / 복구 검증

기본적으로 정상 - 변경 - 복구 검증의 과정은 다음 표와 같은 단계를 거쳤고, 결과 및 증빙은 다음과 같다.

단계	RTL 상태	결과 및 증빙
정상	원래 코드	PASS 로그, 정상 파형
변경	지정 부분 의도적 변경	FAIL 로그 및 FAIL 사진
복구	원상 복구	최종 PASS 및 정상 소스

실험	의도적 변경
11 Counter	증가량 +1 → +2
12 Divider	DIVISOR/2-1 → DIVISOR/2
13 Register	value <= stored → value <= data_in
14 Shift	{serial_in,value[3:1]} 방향 반전
15 PISO	serial_out=value[3] → value[0]

16 Moore	S1의 다음 상태 S2 → S0
17 Mealy	state 0/1의 출력 10과 01 교환
18 Scan	blank일 때 select=0 조건 제거
18A Integrated	Counter 증가량 +1 → +2

실패 FAIL 기록

```

LAB2_FAIL up including 15 to 8 time=16000
FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_01_counter\sin\lab2_counter4.sv:44: check failed
Time: 16000 Scope: tb_counter4.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_01_counter\build\sin-run-98443cb2171745ef83841a69b09664\simulation.log
  
```

그림 29: 실험 11 시뮬레이션 실행 결과(FAIL)

```

LAB2_FAIL clock divider time=16000
FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_02_clock_divider\sin\lab2_clock_divider.sv:42:
Time: 16000 Scope: tb_clock_divider.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_02_clock_divider\build\sin-run-c5071ee34ec4228b3baaded125746a5\simulation.log
  
```

그림 30: 실험 12 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_03_register\sin\lab2_register_pair.sv:47: check failed
Time: 26000 Scope: tb_register_pair.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_03_register\build\sin-run-e4e208ef7194af78b2a62257f829c84\simulation.log
  
```

그림 31: 실험 13 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_04_shift_register\sin\lab2_shift_register4.sv:43: check failed
Time: 16000 Scope: tb_shift_register4.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_04_shift_register\build\sin-run-8047fc93a674b3a6a367d8cccc680c\simulation.log
  
```

그림 32: 실험 14 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_05_piso\sin\lab2_piso4.sv:31:
Time: 86000 Scope: tb_piso4.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_05_piso\build\sin-run-2559281164a42eb34a3ee8b0b77a0b\simulation.log
  
```

그림 33: 실험 15 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_06_moore\sin\lab2_moore_cycle.sv:44: check failed
Time: 46000 Scope: tb_moore_cycle.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_06_moore\build\sin-run-0943e71c27a487a794ed054b752f8\simulation.log
  
```

그림 34: 실험 16 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_07_mealy\sin\lab2_mealy_toggle.sv:45: check failed
Time: 7000 Scope: tb_mealy_toggle.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_07_mealy\build\sin-run-ab71018c4351424d8888bea2b20fa36c\simulation.log
  
```

그림 35: 실험 17 시뮬레이션 실행 결과(FAIL)

```

FATAL: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_08_segment_scan\sin\lab2_segment_scan0.sv:33:
Time: 6000 Scope: tb_segment_scan0.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ece2-lab2\lab2_08_segment_scan\build\sin-run-27aeb68ec5244b9a8934f8072f6875b\simulation.log
  
```

그림 36: 실험 18 시뮬레이션 실행 결과(FAIL)

```
LAB2_INTEGRATED_PASS counter increments once time=346000
FATAL: D:\Engineering\Verilog_Lab\ex2-lab2\lab2_08a_integrated(sim)\tb_lab2_integrated.sv:23: check failed
Time: 346000 Scope: tb_lab2_integrated.check
ERROR: 종료 코드 1: D:\Engineering\Verilog_Lab\ex2-lab2\lab2_08a_integrated\build\sim\rn-8691e8d6e6645923b03ba81cf06cc54\simulation.log
```

그림 37: 실험 18A 시뮬레이션 실행 결과(FAIL)

최종 정상 코드 커밋 및 PASS 증빙

실험	실패 · 복구 기록
11	최종: 3176bd0..., 최종로그
12	최종: 9575ae4..., 최종로그
13	최종: 39f6275..., 최종로그
14	최종: b74a7d1..., 최종로그
15	최종: e731f55..., 최종로그
16	최종: ad2cdc8..., 최종로그
17	최종: 8b0d8cb..., 최종로그
18	최종: 3739919..., 최종로그
18A	최종: 0b7f569..., 최종로그

14. 보드 검증 계획

실제 실험에서는 Vivado에서 합성 및 구현을 수행한 뒤 FPGA 보드를 프로그래밍 한다. 이후 스위치와 버튼으로 입력을 변경하여 순차회로 각각의 출력을 확인하고 시뮬레이션 결과와 비교한다. 또한 보드 검증 결과는 입력과 출력 장치가 함께 보이는 사진 및 동작 영상으로 기록하고, 예상결과와의 일치 여부를 실험 후 보고서에 정리한다.

15. 결론

LAB2에서는 클록 상승 에지와 이전 상태에 의해 출력이 결정되는 여덟 종류의 순차논리회로를 설계하였다. 테스트 벤치는 reset, enable, 상태 유지, 상태 전이, wrap-around, 제어 신호 우선순위 및 표시장치의 blank 조건을 자동으로 검사하였다. 모든 개별 회로에서 지정된 검사 횟수의 PASS를 확인하였고, 첨부한 VaporView 파형에서 예상한 상태 변화와 실제 출력이 일치함을 확인하였다. 또한 여덟 회로를 하나의 통합 top에 연결하여 모드 선택, 모드 변경 시 회로 초기화, LED 출력 선택, LCD 버스 및 7세그먼트 제어를 검사하였으며 LAB2_INTEGRATED_PASS modes=8 checks=2848 결과를 확인하였다. 실제 실습에서는 Vivado에서 동일한 RTL, TB, XDC를 등록하여 시뮬레이션, 합성, 구현, 타이밍도 등의 분석을 수행하고, FPGA 보드에서 검증할 예정이다.

참고 문헌

[1] E. F. Moore, "Gedanken-Experiments on Sequential Machines," in *Automata Studies*, C. E. Shannon and J. McCarthy, Eds., Annals of Mathematics Studies, no. 34, Princeton

University Press, pp. 129–153, 1956.

[2] G. H. Mealy, "A Method for Synthesizing Sequential Circuits," *The Bell System Technical Journal*, vol. 34, no. 5, pp. 1045–1079, Sep. 1955, doi: 10.1002/j.1538-7305.1955.tb03788.x.